

Secure RBAC-Enabled Medical RAG Assistant — Project Guide

Pratham Bhat · private interview pack · 2026-09-09 · DOCX cheat sheet is local-only

EDUCATIONAL / RESEARCH ONLY — NOT MEDICAL ADVICE. This system answers from uploaded PDFs with role-based access control. It is not a regulated clinical decision support system, not HIPAA-certified, and must not be used for real patient care without proper governance.

1. What this project is

A healthcare-oriented Retrieval-Augmented Generation (RAG) chatbot with **role-based access control (RBAC)**. Admins and doctors upload PDF clinical or educational documents tagged by visibility role. Nurses, patients, and guests query only the chunks their role is allowed to see. The stack is FastAPI + Streamlit, Supabase PostgreSQL for users, Pinecone for vectors, local Ollama for embeddings, and Groq for LLM inference.

What it is not

- Not a replacement for Epic/Cerner or an EHR integration.
- Not HIPAA-compliant out of the box (demo auth, no audit trail, no PHI encryption story).
- Not a fine-tuned medical LLM — answers are grounded in retrieved chunks only.
- Not a substitute for a clinician's judgment.

2. End-to-end architecture

Ingest path: Admin/Doctor uploads PDF → PyPDFLoader → RecursiveCharacterTextSplitter (500 chars, 50 overlap) → Ollama nomic-embed-text (768-d) → Pinecone upsert with metadata: source, doc_id, role, page, text.

Query path: User authenticates (HTTP Basic) → role resolved from Postgres → query embedded locally → Pinecone similarity search with **metadata filter on role** → top-5 chunks concatenated → Groq LLM with strict context-only prompt → answer + sources.

The critical design fix: RBAC is enforced **at retrieval time** via Pinecone `filter`, not by filtering after top-k. Post-filtering let unauthorized documents consume the k slots and produced false 'no results' for lower-privilege roles even when allowed docs existed.

3. Role hierarchy

Role	Can upload	Can retrieve chunks tagged
admin	Yes	all roles
doctor	Yes	doctor, nurse, patient, other
nurse	No	nurse, patient, other
patient	No	patient, other
other	No	other only

4. Design decisions — why this, not that

Choice	Rejected alternatives	Why we kept this
Pinecone metadata filter (pre-retrieval RBAC)	Post-filter top-k in Python	Post-filter is a classic RAG security bug: top-5 may be all admin-only, so nurses see nothing. Filter shrinks the searchable set before ranking.
Ollama local embeddings	OpenAI embeddings, HuggingFace API, in-process Torch	Privacy (clinical text stays local), zero embedding API cost, no Windows DLL issues from heavy ML stacks. nomic-embed-text = 768-d.
Groq cloud LLM	Local llama3.2 3B only, GPT-4	Fast inference for demo UX. Generation is not the PHI-heavy step if embeddings are local. Model default: openai/gpt-oss-120b (Groq retired llama-3.3-70b).

Supabase PostgreSQL	MongoDB Atlas	Relational users table, CHECK constraint on role, case-insensitive username index, connection pooling. Session pooler works on IPv4-only networks.
HTTP Basic Auth + bcrypt	JWT, OAuth, API keys	Simplest correct demo for a portfolio project. bcrypt for password storage. JWT would be the production upgrade (stateless, expiry, refresh).
FastAPI backend + Streamlit client	Streamlit-only monolith, React SPA	Separates API contract from UI. Streamlit is fast to ship for healthcare demos.
Chunk size 500 / overlap 50	Larger 2k chunks, semantic chunking	Medical PDFs mix headers and bullets; smaller chunks improve precision for specific symptom/drug questions at the cost of broader context per chunk.
Cosine metric in Pinecone	Dot product (old index)	Matches normalized embedding geometry for nomic-embed-text; index dimension must be 768.

5. Data stores and responsibilities

- **Supabase Postgres:** users (username, bcrypt hash, role). Auto-created on startup.
- **Pinecone:** vector index + metadata (text, source, role, page, doc_id).
- **Local disk:** uploaded PDF copies under server/uploaded_docs/ (gitignored).
- **Ollama:** embedding runtime only — no persistent vector store.

6. API surface

Method	Route	Auth	Purpose
GET	/health	None	API + DB ping
POST	/signup	None	Register user with validated role
GET	/login	Basic	Authenticate, return role
POST	/upload_docs	Basic (admin/doctor)	PDF → Pinecone
POST	/chat	Basic	Role-aware RAG Q&A;

7. Bugs fixed in the audit pass

- **Post-retrieval RBAC:** moved to Pinecone metadata filter — the main logical error.
- **bcrypt decode typo:** utf=8 → utf-8 in hash_utils (could break password verify).
- **Unvalidated signup role:** any string accepted; now CHECK + Pydantic validator.
- **Doctor upload in UI:** Streamlit only showed upload for admin; API allowed doctors.
- **Destructive Pinecone index delete:** removed auto-delete on dimension mismatch.
- **Retired Groq model:** llama-3.3-70b-versatile → GROQ_MODEL env default.
- **MongoDB → Supabase:** users migrated to Postgres with connection pool.
- **Empty query / empty PDF / path traversal:** validation added on API and client.

8. Pain points we actually hit

- **Supabase direct host is IPv6-only:** db.*.supabase.co failed DNS on IPv4 Windows. Fix: session pooler aws-0-<region>.pooler.supabase.com:5432 with username postgres.<project-ref>.
- **Ollama must be running:** embeddings return empty without nomic-embed-text pulled.
- **Groq model deprecation:** hardcoded model ID 404'd; made configurable via GROQ_MODEL.
- **Streamlit vs API port mismatch:** API_URL must match uvicorn port (8001 in start_app.bat).
- **Scanned PDFs:** PyPDFLoader returns no text — upload fails with clear error.

9. Security and production gaps (volunteer before they ask)

- HTTP Basic sends credentials on every request — use HTTPS in production.
- No rate limiting, audit log, or document deletion API.
- Signup is open — production needs admin-provisioned accounts or email verification.
- LLM prompt says 'healthcare assistant' but has no clinical safety guardrails layer.
- No chunk-level citation UI — sources are filenames only.
- Pinecone and Groq are third-party; data leaves the VPC at inference time.

10. How to run (interview demo)

- Terminal 1: `ollama serve && ollama pull nomic-embed-text`
- Terminal 2: `cd server && uvicorn main:app --reload --port 8001`
- Terminal 3: `streamlit run client/main.py`
- Or: `start_app.bat` on Windows.
- `.env` needs `DATABASE_URL`, `PINECONE_*`, `GROQ_API_KEY`, `API_URL=http://127.0.0.1:8001`

11. Future enhancements (credible roadmap)

- JWT auth + refresh tokens; admin-only signup.
- Hybrid search (BM25 + dense) for exact drug names.
- Audit table: who queried what, which chunks retrieved.
- Document lifecycle: delete/re-index by `doc_id`.
- Citation spans with page numbers in Streamlit UI.

Appendix — key files

server/main.py · server/config/db.py · server/auth/routes.py · server/auth/models.py · server/chat/chat_query.py · server/chat/routes.py · server/docs/routes.py · server/docs/vectorstore.py · server/embeddings/ollama_local.py · client/main.py · start_app.bat · .env.example